

FirstKnock Business Scraper & Contact Database — Complete Technical Blueprint (Sections 1 & 2)

FirstKnock Business Scraper & Contact Database

Complete Technical Blueprint — Sections 1 & 2: Scraping at Scale + Email & Phone Discovery Engine

2025

Target: 20,000+ verified contacts across Pest Control · Solar · Home Security · Roofing · HVAC · D2D Sales

Stack: Python 3.11+ · Playwright/patchright · PostgreSQL · asyncio · dnspython · phonenumbers

20,000+Target Verified Contacts	6Target Verticals	3Scrape Sources	~80%Email Discovery Coverage
---------------------------------	-------------------	-----------------	------------------------------

SECTION 1 — SCRAPING AT SCALE

1A. Source Strategy

Google Maps (Playwright/Headless Chromium)

Approach: Playwright async with stealth patching. Google Maps renders via XHR/JSON internally but the public-facing UI is fully JS-rendered. Direct API is rate-limited and requires billing. Scraping the rendered DOM is the only zero-cost path.

Core selectors (verified 2024-2025):

```
# Search results listRESULTS_CONTAINER= 'div[role="feed"]'RESULT_ITEM= 'div[role="feed"] > div > div > a' # each listing link# Detail panel (after clicking a result)BUSINESS_NAME = 'h1.DUwDvf'RATING = 'div.F7nice > span[aria-hidden="true"]'REVIEW_COUNT = 'div.F7nice > span > span[aria-label] 'CATEGORY = 'button.DkEaL'ADDRESS = 'button[data-item-id="address"] div.lo6YTePHONE = 'button[data-item-id^="phone"] div.lo6YTeWEBSITE = 'a[data-item-id="authority"] div.lo6YTeHOURS_BUTTON = 'div[data-hide-tooltip-on-mouse-move] > div > div'PLUS_CODE = 'button[data-item-id="oloc"] div.lo6YTe'
```

Extractable fields: Business name, address, phone, website URL, rating, review count, category, lat/lng (from URL), hours, plus code, photos count, owner response presence.

Rate limits / detection thresholds:

- Google aggressively fingerprints: canvas, WebGL, font enumeration, navigator.webdriver
- Soft block at ~150-200 requests/IP/hour without stealth

- Hard CAPTCHA at ~50 requests/IP/hour on datacenter IPs
- Residential proxies: ~300-500 req/IP/day before soft throttle
- Session rotation every 50-75 requests is safe

Core scrape loop:

```
import asyncioimport randomfrom playwright.async_api import async_playwrightfrom urllib.parse
import quoteSEARCH_URL = "https://www.google.com/maps/search/{query}/{lat},{lng},13z"
async def scrape_gmaps_zip(query: str, zip_code: str, proxy: dict) -> list[dict]:"""Scrape Google
Maps for a query+ZIP. Returns list of business dicts.proxy: {"server": "http://host:port", "username":
"u", "password": "p"}"""results = []async with async_playwright() as p:browser = await
p.chromium.launch(headless=Trueproxy=proxy,args=[
"--disable-blink-features=AutomationControlled","--no-sandbox","--disable-dev-shm-usage",
"--disable-gpu","--window-size=1366,768".])context = await browser.new_context(
viewport={"width": 1366, "height": 768},user_agent=get_random_ua(),locale="en-US",
timezone_id="America/Chicago",geolocation={"latitude": 41.85, "longitude": -87.65},
permissions=["geolocation"],)# Inject stealth JS before any navigationawait
context.add_init_script("""Object.defineProperty(navigator, 'webdriver', {get: () => undefined});
Object.defineProperty(navigator, 'plugins', {get: () => [1,2,3,4,5]});Object.defineProperty(navigator,
'languages', {get: () => ['en-US','en']});window.chrome = {runtime: {}};""")page = await
context.new_page()search_query = quote(f"{query} {zip_code}")await
page.goto(f"https://www.google.com/maps/search/{search_query}",wait_until="networkidle")await
asyncio.sleep(random.uniform(2.0, 4.0))# Scroll the results feed to load all listingsfeed =
page.locator('div[role="feed"]')prev_count = 0for _ in range(15): # max 15 scroll attemptsitems =
await page.locator('div[role="feed"] > div > div > a').all()if len(items) == prev_count:break
prev_count = len(items)await feed.evaluate("el => el.scrollBy(0, 800)")await
asyncio.sleep(random.uniform(1.2, 2.5))# Collect all listing URLs first, then visit eachlinks = await
page.locator('div[role="feed"] > div > div > a').evaluate_all("els => els.map(e => e.href)")for link in
links[:20]: # cap at 20 per ZIP/query combotry:detail = await scrape_gmaps_detail(context, link)if
detail:results.append(detail)await asyncio.sleep(random.uniform(1.5, 3.5))except Exception as e:
continueawait browser.close()return resultsasync def scrape_gmaps_detail(context, url: str) -> dict
| None:page = await context.new_page()try:await page.goto(url, wait_until="networkidle",
timeout=30000)await asyncio.sleep(random.uniform(1.0, 2.0))def safe_text(selector):return
page.locator(selector).first.inner_text()name = await page.locator("h1.DUwDvf").first.inner_text()
phone = ""website = ""address = ""category = ""try:phone = await
page.locator('button[data-item-id^="phone"] div.lo6YTe').first.inner_text()except: passtry:website =
await page.locator('a[data-item-id="authority"] div.lo6YTe').first.inner_text()except: passtry:address
= await page.locator('button[data-item-id="address"] div.lo6YTe').first.inner_text()except: passtry:
category = await page.locator('button.DkEaL').first.inner_text()except: pass# Extract lat/lng from
current URLlat, lng = extract_latlng(page.url)return {"name": name.strip(),"phone": phone.strip(),
"website": website.strip(),"address": address.strip(),"category": category.strip(),"lat": lat,"lng": lng,
"source_url": url,"source": "google_maps"},except Exception as e:return Nonefinally:await
```

```

page.close()
def extract_latlng(url: str):
    import re
    m = re.search(r'@(-?\d+\.\d+),(-?\d+\.\d+)', url)
    if m:
        return float(m.group(1)), float(m.group(2))
    return None, None

```

Yelp

Approach: requests + BeautifulSoup. Yelp's search results are server-side rendered (SSR) with embedded JSON-LD and a `__NEXT_DATA__` script tag containing structured business data. This is far more reliable than CSS scraping.

Core selectors / extraction:

```

import requests
from bs4 import BeautifulSoup
import json, re
HEADERS = {"User-Agent":
get_random_ua(), "Accept-Language": "en-US,en;q=0.9", "Accept":
"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8", "Referer": "https://www.yelp.com/",
}
def scrape_yelp_search(category: str, location: str, session: requests.Session) -> list[dict]:
    """
    category: e.g. 'pest-control', 'solar-installation', 'roofing'
    location: ZIP code string
    """
    results = []
    for offset in range(0, 240, 30): # Yelp caps at 240 results per search
        url = f"https://www.yelp.com/search?find_desc={category}&find_loc={location}&start={offset}"
        resp = session.get(url, headers=HEADERS, timeout=15)
        if resp.status_code == 429:
            raise RateLimitError("Yelp 429")
        soup = BeautifulSoup(resp.text, "lxml")
        # Primary: extract from __NEXT_DATA__ JSON blob
        script = soup.find("script", id="__NEXT_DATA__")
        if script:
            data = json.loads(script.string)
            try:
                biz_list = data["props"]["pageProps"]["searchPageProps"]["mainContentComponentsListProps"]
                for item in biz_list:
                    if item.get("componentKey") == "bizListItem":
                        biz = item.get("props", {}).get("bizListItemProps", {})
                        results.append({
                            "name": biz.get("name"),
                            "phone": biz.get("phone"),
                            "address": biz.get("formattedAddress"),
                            "rating": biz.get("rating"),
                            "review_count": biz.get("reviewCount"),
                            "categories": [c["title"] for c in biz.get("categories", [])],
                            "yelp_url": "https://www.yelp.com" + biz.get("businessUrl", ""),
                            "website": biz.get("website", {}).get("href", "source": "yelp",
                        })
            except (KeyError, TypeError):
                pass # Fall through to CSS fallback
        # CSS fallback if JSON structure changes
        if not results:
            for card in soup.select('div[data-testid="serp-ia-card"]'):
                name_el = card.select_one('a.css-19v1rkv')
                phone_el = card.select_one('p.css-1p9ibgf')
                addr_el = card.select_one('address')
                results.append({
                    "name": name_el.text.strip() if name_el else None,
                    "phone": phone_el.text.strip() if phone_el else None,
                    "address": addr_el.text.strip() if addr_el else None,
                    "source": "yelp",
                })
            import time, random
            time.sleep(random.uniform(3.0, 6.0)) # Yelp is aggressive; 3-6s minimum
    return results

```

Rate limits: Yelp blocks datacenter IPs almost immediately. Residential proxies required. 1 request/3-6s per IP. Rotate IP every 10-15 requests. Yelp uses Akamai Bot Manager — TLS fingerprinting is a factor; use `curl_cffi` or `tls-client` instead of `requests` for better bypass.

Extractable fields: Business name, phone, address, rating, review count, categories, Yelp URL, website URL, hours (on detail page), owner name (Meet the Owner section).

Yellow Pages (YP.com)

Approach: requests + BeautifulSoup. YP.com is largely SSR with minimal JS. Easier to scrape than

Google/Yelp but lower data quality and more duplicates.

```
def scrape_yellowpages(category: str, location: str, session: requests.Session) -> list[dict]:"""
category: e.g. 'pest+control', 'solar+energy', 'roofing+contractors'location: ZIP code"""results = []for
page_num in range(1, 6): # YP shows ~30/page, cap at 5 pagesurl =
f"https://www.yellowpages.com/search?search_terms={category}&geo_location_terms={location}&p
age={page_num}"resp = session.get(url, headers=HEADERS, timeout=15)soup =
BeautifulSoup(resp.text, "lxml")listings = soup.select('div.result > div.info')if not listings:breakfor
listing in listings:name_el = listing.select_one('a.business-name')phone_el =
listing.select_one('div.phones.phone.primary')addr_el = listing.select_one('div.adr')website_el =
listing.select_one('a.track-visit-website')cats_el = listing.select('div.categories a')results.append({
"name": name_el.text.strip() if name_el else None,"phone": phone_el.text.strip() if phone_el else
None,"address": addr_el.text.strip() if addr_el else None,"website": website_el.get("href") if
website_el else None,"categories": [c.text for c in cats_el],"yp_url": "https://www.yellowpages.com"
+ name_el.get("href", "") if name_el else None,"source": "yellowpages",})import time, random
time.sleep(random.uniform(2.0, 4.0))return results
```

Rate limits: YP.com is relatively lenient — datacenter proxies work at 1 req/2-4s. Rotate UA every request. No CAPTCHA observed below 100 req/IP/hour.

Extractable fields: Business name, phone, address, website, categories, YP listing URL, years in business (sometimes), accreditation badges.

1B. Geographic Coverage Strategy

Why ZIP-level beats city/state-level

Google Maps and Yelp cap results at 20 (Maps) and 240 (Yelp) per search query. A city-level search for "pest control Chicago" returns 20 results — but Chicago has 60+ ZIP codes, each potentially containing 10-20 unique businesses. ZIP-level queries multiply coverage by 10-30x in dense metros. Additionally, ZIP-level allows precise deduplication (same business appearing in multiple ZIPs is caught by phone/address normalization), and enables geographic load distribution across proxy pools.

Scale math:

- 42,741 US ZIP codes (Census Bureau) times 6 verticals = 256,446 queries
- Google Maps: ~15 results/query avg = 3.8M raw records (before dedup)
- After dedup (~60% overlap): ~1.5M unique businesses
- Target 20,000 verified contacts = top 1.3% by data completeness

Free ZIP Code Sources

1. US Census Bureau ZCTA file —
<https://www.census.gov/geographies/reference-files/time-series/geo/gazetteer-files.html>

2. Download: 2023_Gaz_zcta_national.zip
3. Fields: GEOID (ZIP), ALAND, AWATER, INTPTLAT, INTPTLONG
4. 33,791 ZCTAs (ZIP Code Tabulation Areas) — covers all populated areas
5. simplemaps.com/data/us-zips — Free tier: 33,000+ ZIPs with city, state, county, lat/lng, population, timezone
6. Direct CSV download, no auth required
7. URL:
https://simplemaps.com/static/data/us-zips/1.82/basic/simplemaps_uszips_basicv1.82.zip

Loading ZIPs:

```
import pandas as pd
def load_zip_codes(csv_path: str = "uszips.csv") -> pd.DataFrame:
    df = pd.read_csv(csv_path, dtype={"zip": str})
    # Pad to 5 digits
    df["zip"] = df["zip"].str.zfill(5)
    # Filter to continental US + Hawaii + Alaska (exclude territories)
    df = df[df["state_id"].isin(["AL", "AK", "AZ", "AR", "CA", "CO", "CT", "DE", "FL", "GA", "HI", "ID", "IL", "IN", "IA", "KS", "KY", "LA", "ME", "MD", "MA", "MI", "MN", "MS", "MO", "MT", "NE", "NV", "NH", "NJ", "NM", "NY", "NC", "ND", "OH", "OK", "OR", "PA", "RI", "SC", "SD", "TN", "TX", "UT", "VT", "VA", "WA", "WV", "WI", "WY", "DC"])]
    return df[["zip", "city", "state_id", "lat", "lng", "population"]].reset_index(drop=True)
```

PostgreSQL-backed Job Queue

```
-- schema.sql
CREATE TABLE scrape_jobs (
    id BIGSERIAL PRIMARY KEY,
    zip_code CHAR(5) NOT NULL,
    vertical VARCHAR(50) NOT NULL,
    source VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    attempts SMALLINT NOT NULL DEFAULT 0,
    last_error TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    started_at TIMESTAMPTZ,
    finished_at TIMESTAMPTZ,
    result_count INT,
    UNIQUE(zip_code, vertical, source)
);
CREATE INDEX idx_scrape_jobs_status ON scrape_jobs(status);
CREATE INDEX idx_scrape_jobs_zip ON scrape_jobs(zip_code);
```

```
import asyncio
import asyncpg
from dataclasses import dataclass
from typing import Optional

@dataclass
class ScrapeJob:
    id: int
    zip_code: str
    vertical: str
    source: str

class PGJobQueue:
    def __init__(self, dsn: str):
        self.dsn = dsn
        self.pool: Optional[asyncpg.Pool] = None

    async def connect(self):
        self.pool = await asyncpg.create_pool(self.dsn, min_size=2, max_size=10)

    async def seed_jobs(self, zip_codes: list[str], verticals: list[str], sources: list[str]):
        """Idempotent — skips existing rows due to UNIQUE constraint."""
        async with self.pool.acquire() as conn:
            await conn.executemany(
                """INSERT INTO scrape_jobs (zip_code, vertical, source) VALUES ($1, $2, $3) ON CONFLICT DO NOTHING""",
                [(z, v, s) for z in zip_codes for v in verticals for s in sources]
            )

    async def claim_job(self, worker_id: str) -> Optional[ScrapeJob]:
        """Atomically claim one pending job using SKIP LOCKED."""
        async with self.pool.acquire() as conn:
            row = await conn.fetchrow(
                """UPDATE scrape_jobs SET status = 'running', started_at = NOW(), attempts = attempts + 1 WHERE id = (SELECT id FROM scrape_jobs WHERE status = 'pending' OR (status = 'failed' AND attempts < 3) ORDER BY id LIMIT 1 FOR UPDATE SKIP LOCKED) RETURNING id, zip_code, vertical, source"""
            )
            if row:
                return ScrapeJob(*row)
```

```

ScrapeJob(**dict(row))      return None
async def complete_job(self, job_id: int, result_count: int):
    with self.pool.acquire() as conn:
        await conn.execute("""UPDATE scrape_jobs SET status = 'done',
        finished_at = NOW(), result_count = $2 WHERE id = $1""", job_id, result_count)
    async def fail_job(self, job_id: int, error: str):
        with self.pool.acquire() as conn:
            await conn.execute("""UPDATE scrape_jobs SET status = 'failed',
            finished_at = NOW(), last_error = $2 WHERE id = $1""", job_id, error)
    async def stats(self) -> dict:
        with self.pool.acquire() as conn:
            rows = await conn.fetch("""SELECT status, COUNT(*) as cnt FROM scrape_jobs GROUP BY status""")
        return {r["status"]: r["cnt"] for r in rows}

```

Worker pool:

```

async def worker(queue: PGJobQueue, scraper_fn, concurrency_sem: asyncio.Semaphore):
    while True:
        job = await queue.claim_job("worker")
        if not job:
            await asyncio.sleep(5)
            continue
        async with concurrency_sem:
            try:
                proxy = proxy_manager.get_proxy()
                results = await scraper_fn(job.vertical, job.zip_code, proxy)
                await save_results(results)
                await queue.complete_job(job.id, len(results))
            except Exception as e:
                await queue.fail_job(job.id, str(e))
    async def run_workers(dsn: str, n_workers: int = 10):
        queue = PGJobQueue(dsn)
        await queue.connect()
        sem = asyncio.Semaphore(n_workers)
        tasks = [asyncio.create_task(worker(queue, scrape_gmaps_zip, sem))
        for _ in range(n_workers)]
        await asyncio.gather(*tasks)

```

Realistic time estimates:

Table 1 — Worker Concurrency vs. Estimated Completion Time

Concurrency	Req/hr	256K Jobs ETA	Notes
5 workers	~300	~35 days	Single residential IP pool, safe
20 workers	~1,200	~9 days	20+ residential proxies needed
50 workers	~3,000	~3.5 days	50+ proxies, monitor block rate
100 workers	~6,000	~1.8 days	Requires proxy rotation infra

Assumes 4-6s avg per request including page load + parsing. Google Maps detail pages add 2-3s each.

1C. Anti-Bot Detection & Evasion

Proxy Rotation Manager

```

import asyncio
import aiohttp
import random
import time
from dataclasses import dataclass, field
from typing import Optional
import logging
logger = logging.getLogger(__name__)

@dataclass
class Proxy:

```

```

server: str # "http://host:port"
username: str = ""
password: str = ""
failures: int = 0
last_used: float = 0.0
last_checked: float = 0.0
alive: bool = True
requests_made: int = 0
def to_playwright_dict(self) -> dict:
    d = {"server": self.server}
    if self.username:
        d["username"] = self.username
    if self.password:
        d["password"] = self.password
    return d
def to_requests_dict(self) -> dict:
    if self.username:
        auth = f"{self.username}:{self.password}@"
        proto = self.server.split("://")[1]
        host = self.server.split("://")[2]
        url = f"{proto}://{auth}{host}"
    else:
        url = self.server
    return {"http": url, "https": url}
class ProxyManager:
    def __init__(self, proxies: list[Proxy], max_failures: int = 3, cooldown_seconds: int = 300, check_url: str = "https://httpbin.org/ip", min_delay_between_uses: float = 1.0):
        self.proxies = proxies
        self.max_failures = max_failures
        self.cooldown = cooldown_seconds
        self.check_url = check_url
        self.min_delay = min_delay_between_uses
        self._lock = asyncio.Lock()
    @classmethod
    def from_list(cls, proxy_strings: list[str], **kwargs) -> ProxyManager:
        """proxy_strings format: "http://user:pass@host:port" or "http://host:port"
        """
        proxies = []
        for s in proxy_strings:
            if "@" in s:
                proto_auth, hostport = s.rsplit("@", 1)
                proto, auth = proto_auth.split("://", 1)
                user, pwd = auth.split(":", 1)
                proxies.append(Proxy(server=f"{proto}://{hostport}", username=user, password=pwd))
            else:
                proxies.append(Proxy(server=s))
        return cls(proxies, **kwargs)
    async def get_proxy(self) -> Optional[Proxy]:
        async with self._lock:
            now = time.time()
            candidates = [p for p in self.proxies if p.alive and p.failures < self.max_failures and (now - p.last_used) >= self.min_delay]
            if not candidates:
                # Try to revive cooled-down proxies
                revivable = [p for p in self.proxies if not p.alive and (now - p.last_checked) > self.cooldown]
                if revivable:
                    proxy = random.choice(revivable)
                    proxy.alive = True
                    proxy.failures = 0
            return proxy
        return None
    # Caller must wait
    # Weighted selection: prefer proxies with fewer failures and longer idle time
    weights = [(self.max_failures - p.failures + 1) * (now - p.last_used + 1) for p in candidates]
    proxy = random.choices(candidates, weights=weights, k=1)[0]
    proxy.last_used = now
    proxy.requests_made += 1
    return proxy
    async def mark_failure(self, proxy: Proxy, error: str = ""):
        async with self._lock:
            proxy.failures += 1
            proxy.last_checked = time.time()
            if proxy.failures >= self.max_failures:
                proxy.alive = False
                logger.warning(f"Proxy {proxy.server} marked dead after {proxy.failures} failures. Last error: {error}")
    async def mark_success(self, proxy: Proxy):
        async with self._lock:
            proxy.failures = max(0, proxy.failures - 1)
    async def health_check_all(self, timeout: int = 10):
        """Run health checks on all proxies concurrently.
        """
    async def check(proxy: Proxy):
        try:
            async with aiohttp.ClientSession() as session:
                async with session.get(self.check_url, proxy=proxy, proxy_auth=aiohttp.BasicAuth(proxy.username, proxy.password)) if proxy.username else None, timeout=aiohttp.ClientTimeout(total=timeout) as resp:
                    if resp.status == 200:
                        proxy.alive = True
                        proxy.failures = 0
                    else:
                        proxy.alive = False
        except Exception:
            proxy.alive = False
            proxy.last_checked = time.time()
        await asyncio.gather(*[check(p) for p in self.proxies])
    def stats(self) -> dict:
        alive = sum(1 for p in self.proxies if p.alive)
        return {"total": len(self.proxies), "alive": alive, "dead": len(self.proxies) - alive, "total_requests": sum(p.requests_made for p in self.proxies),}

```

Proxy sources:

- Residential (paid, required for Google/Yelp): Bright Data (~\$8.4/GB), Oxylabs (~\$8/GB), Smartproxy (~\$7/GB), IPRoyal (~\$3/GB)
- Datacenter (OK for YP.com): Webshare.io (free tier: 10 proxies), ProxyScrape.com (free list,

high failure rate)

- Free residential (unreliable, use only for testing): ProxyList.to, free-proxy-list.net — expect 70%+ failure rate

User Agent Strings (2024-2025)

```
import random# Weighted UA pool — weights reflect real-world browser market share
USER_AGENTS = [# Chrome on Windows (dominant — ~65% weight)("Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36", 12),
("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/130.0.0.0 Safari/537.36", 10),("Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0 Safari/537.36", 8),("Mozilla/5.0
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0
Safari/537.36", 6),("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/127.0.0.0 Safari/537.36", 4),# Chrome on macOS("Mozilla/5.0 (Macintosh; Intel
Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36",
8),("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/130.0.0.0 Safari/537.36", 6),("Mozilla/5.0 (Macintosh; Intel Mac OS X 14_7_1)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36", 5),# Firefox on
Windows("Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:132.0) Gecko/20100101 Firefox/132.0",
6),("Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:131.0) Gecko/20100101 Firefox/131.0", 4),
("Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:130.0) Gecko/20100101 Firefox/130.0", 3),#
Firefox on macOS("Mozilla/5.0 (Macintosh; Intel Mac OS X 14.7; rv:132.0) Gecko/20100101
Firefox/132.0", 4),("Mozilla/5.0 (Macintosh; Intel Mac OS X 14.7; rv:131.0) Gecko/20100101
Firefox/131.0", 3),# Safari on macOS("Mozilla/5.0 (Macintosh; Intel Mac OS X 14_7_1)
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.0 Safari/605.1.15", 5),("Mozilla/5.0
(Macintosh; Intel Mac OS X 14_6) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/17.6
Safari/605.1.15", 3),# Edge on Windows("Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36 Edg/131.0.0.0", 4),
("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/130.0.0.0 Safari/537.36 Edg/130.0.0.0", 3),# Chrome on Linux (lower weight — less
common for consumers)("Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/131.0.0.0 Safari/537.36", 2),("Mozilla/5.0 (X11; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36", 2),# Chrome on
Android (mobile — useful for Yelp mobile endpoint)("Mozilla/5.0 (Linux; Android 14; Pixel 8)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.6778.39 Mobile Safari/537.36", 3),
("Mozilla/5.0 (Linux; Android 13; SM-S918B) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/131.0.0.0 Mobile Safari/537.36", 2),# Safari on iPhone("Mozilla/5.0 (iPhone; CPU iPhone
OS 18_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.1 Mobile/15E148
Safari/604.1", 3),]_ua_strings = [ua for ua, _ in USER_AGENTS]_ua_weights = [w for _, w in
USER_AGENTS]def get_random_ua() -> str:return random.choices(_ua_strings,
```

```
weights=_ua_weights,      k=1)[0]
```

Request Timing with numpy Normal Distribution

```
import numpy as npimport asyncioclass HumanDelay: """Models human inter-request timing using
a truncated normal distribution.Different profiles for different scraping contexts."""PROFILES = {
"fast_reader": {"mean": 2.5, "std": 0.8, "min": 1.0, "max": 6.0}, "normal_reader": {"mean": 4.5, "std":
1.5, "min": 2.0, "max": 10.0}, "slow_reader": {"mean": 8.0, "std": 2.5, "min": 4.0, "max": 20.0},
"page_load": {"mean": 1.5, "std": 0.5, "min": 0.5, "max": 4.0}, "between_pages": {"mean": 3.0, "std":
1.0, "min": 1.5, "max": 8.0},} @staticmethoddef sample(profile: str = "normal_reader") -> float: p =
HumanDelay.PROFILES[profile]while True: val = np.random.normal(p["mean"], p["std"])if p["min"]
<= val <= p["max"]:return float(val) @staticmethoddef async def wait(profile: str = "normal_reader"):
delay = HumanDelay.sample(profile)await asyncio.sleep(delay) @staticmethoddef
add_micro_jitter(base_delay: float, jitter_pct: float = 0.15) -> float: """Add plusminus15% jitter to any
base delay."""jitter = base_delay * jitter_pct * np.random.uniform(-1, 1)return max(0.1, base_delay
+ jitter)
```

Playwright Stealth Configuration

Note on library status: Atubodad/playwright_stealth is partially broken with Playwright 1.45+. Use the maintained fork Mattwmaster58/playwright_stealth or switch to patchright (<https://github.com/Kaliiiiiiiii-Vinyzu/patchright-python>) which patches Chromium at the binary level — the most robust option in 2025.

```
# Option A: patchright (recommended — drop-in Playwright replacement)# pip install patchright &&
patchright install chromiumfrom patchright.async_api import async_playwright as
patched_playwrightasync def create_stealth_browser(proxy: dict):async with patched_playwright()
as p:browser = await p.chromium.launch(headless=Trueproxy=proxy, args=[
"--disable-blink-features=AutomationControlled", "--no-sandbox", "--disable-setuid-sandbox",
"--disable-dev-shm-usage", "--disable-accelerated-2d-canvas", "--no-first-run", "--no-zygote",
"--disable-gpu", "--window-size=1366,768", "--disable-extensions", "--disable-background-networking",
"--disable-default-apps", "--disable-sync", "--disable-translate", "--hide-scrollbar",
"--metrics-recording-only", "--mute-audio", "--safebrowsing-disable-auto-update"])return browser#
```

```
Option B: Mattwmaster58 fork of playwright_stealth# pip install
git+https://github.com/Mattwmaster58/playwright_stealth.gitfrom playwright_stealth import
stealth_async, StealthConfigasync def apply_stealth(page, ua: str):config = StealthConfig(
navigator_languages=True, navigator_user_agent=True, navigator_vendor=True, webdriver=True,
chrome_app=True, chrome_csi=True, chrome_load_times=True, chrome_runtime=True,
iframe_content_window=True, media_codecs=True, puterdimensions=True, hairline=True)await
stealth_async(page, config)# Override UA after stealth (stealth may set its own)await
page.set_extra_http_headers({"User-Agent": ua, "Accept-Language": "en-US,en;q=0.9",
"Accept-Encoding": "gzip, deflate, br", "sec-ch-ua": "Chromium";v="131", "Not_A_Brand";v="24",
```

```
"Google Chrome";v="131","sec-ch-ua-mobile": "?0","sec-ch-ua-platform": "Windows";})
```

Cookie / Session Management

```
import jsonimport osfrom pathlib import Pathclass SessionManager: """Persist and reuse browser cookies per proxy/domain to maintain session continuity Avoids re-solving CAPTCHAs and reduces fingerprint churn. """def __init__(self, storage_dir: str = "./sessions"): self.storage_dir = Path(storage_dir)self.storage_dir.mkdir(exist_ok=True)def _session_path(self, proxy_id: str, domain: str) -> Path: safe_id = proxy_id.replace(":", "_").replace("/", "_").replace("@", "_")return self.storage_dir / f"{safe_id}_{domain}.json"async def save_cookies(self, context, proxy_id: str, domain: str): cookies = await context.cookies()path = self._session_path(proxy_id, domain)with open(path, "w") as f: json.dump(cookies, f)async def load_cookies(self, context, proxy_id: str, domain: str) -> bool: path = self._session_path(proxy_id, domain)if not path.exists(): return Falsewith open(path) as f: cookies = json.load(f)if cookies: await context.add_cookies(cookies)return Truereturn Falsedef clear_session(self, proxy_id: str, domain: str): path = self._session_path(proxy_id, domain)if path.exists(): path.unlink()def session_age_hours(self, proxy_id: str, domain: str) -> float: path = self._session_path(proxy_id, domain)if not path.exists(): return float("inf")return (time.time() - path.stat().st_mtime) / 3600
```

CAPTCHA Detection + 2captcha Integration

```
import asyncioimport aiohttpimport base64import reclass CaptchaHandler: """Detects and solves reCAPTCHA v2/v3 and image CAPTCHAs via 2captcha. Pricing: ~$0.5-$1.00 per 1000 reCAPTCHA v2 solves (~$0.001/captcha). API docs: https://2captcha.com/api-docs """RECAPTCHA_PATTERNS= [r'google\.com/recaptcha', r'grecaptcha\.execute', r'data-sitekey', r'g-recaptcha',]HCAPTCHA_PATTERNS= [r'hcaptcha\.com', r'data-hcaptcha-sitekey',]def __init__(self, api_key: str, timeout: int = 120): self.api_key = api_keyself.timeout = timeoutself.base_url = "https://2captcha.com"async def detect_captcha(self, page) -> str | None: """Returns 'recaptcha', 'hcaptcha', 'image', or None. """content = await page.content()for pattern in self.RECAPTCHA_PATTERNS: if re.search(pattern, content, re.IGNORECASE): return "recaptcha"for pattern in self.HCAPTCHA_PATTERNS: if re.search(pattern, content, re.IGNORECASE): return "hcaptcha"# Check for image CAPTCHAif await page.locator('img[src*="captcha"]').count() > 0: return "image"return Noneasync def get_sitekey(self, page) -> str | None: """Extract reCAPTCHA sitekey from page. """try: el = page.locator('[data-sitekey]').firstreturn await el.get_attribute("data-sitekey")except: # Try extracting from page sourcecontent = await page.content()m = re.search(r'["\']sitekey["\']\s*:\s*["\']([^\"]+)\s*["\']', content)if m: return m.group(1)return Noneasync def solve_recaptcha_v2(self, page_url: str, sitekey: str) -> str | None: """Submit to 2captcha, poll for result. Returns token string. """async with aiohttp.ClientSession() as session: # Submitsubmit_resp = await session.post(f"{self.base_url}/in.php", data={"key": self.api_key, "method": "userrecaptcha", "googlekey": sitekey, "pageurl": page_url, "json": 1,})submit_data = await submit_resp.json()if submit_data.get("status") != 1: raise ValueError(f"2captcha submit failed: {submit_data}")captcha_id = submit_data["request"]# Poll every 5s up to timeoutfor _ in
```

```

range(self.timeout // 5):await asyncio.sleep(5)result_resp = await session.get(
f"{self.base_url}/res.php",params={"key": self.api_key, "action": "get", "id": captcha_id, "json": 1})
result_data = await result_resp.json()if result_data.get("status") == 1:return result_data["request"]if
result_data.get("request") != "CAPCHA_NOT_READY"raise ValueError(f"2captcha error:
{result_data}")return Noneasync def inject_recaptcha_token(self, page, token: str):"""Inject solved
token into page and trigger callback."""await page.evaluate(f"""
document.getElementById('g-recaptcha-response').innerHTML = '{token}';if (typeof
__grecaptcha_cfg !== 'undefined') {{Object.entries(__grecaptcha_cfg.clients).forEach(([key,
client]) => {{const callback = client?.l?.callback || client?.callback;if (typeof callback === 'function')
callback({'token'});}})}}""")

```

Complete PlaywrightScraper Class

```

import asyncioimport randomimport timeimport loggingfrom typing import Optional, Callable, Any
from dataclasses import dataclass, fieldfrom patchright.async_api import async_playwright,
Browser, BrowserContext, Pagelogger = logging.getLogger(__name__)@dataclassclass
ScraperConfig:max_retries: int = 3base_backoff: float = 2.0max_backoff: float = 60.0page_timeout:
int = 30000navigation_timeout: int = 45000captcha_api_key: str = ""session_dir: str = "./sessions"
headless: bool = Trueclass PlaywrightScraper:"""Production-ready Playwright scraper integrating:-
patchright stealth (Chromium-level patching)- Proxy rotation via ProxyManager- UA rotation with
weighted selection- Human-like delays via HumanDelay- Cookie/session persistence via
SessionManager- CAPTCHA detection + 2captcha solving- Retry with exponential backoff + jitter"""
def __init__(self,proxy_manager: ProxyManager,config: ScraperConfig = None,):
self.proxy_manager = proxy_managerself.config = config or ScraperConfig()self.session_mgr =
SessionManager(self.config.session_dir)self.captcha_handler =
CaptchaHandler(self.config.captcha_api_key)if self.config.captcha_api_key else None
self._playwright = Nonelf._browser: Optional[Browser] = Nonelf._context:
Optional[BrowserContext] = Nonelf._current_proxy: Optional[Proxy] = None
self._requests_on_context: int = 0self._max_requests_per_context: int = random.randint(45, 75)
async def __aenter__(self):self._playwright = await async_playwright().start()await
self._new_browser()return selfasync def __aexit__(self, *args):await self._cleanup()await
self._playwright.stop()async def _new_browser(self):"""Spin up a fresh browser+context with a new
proxy and UA."""if self._browser:try:await self._browser.close()except:passself._current_proxy =
await self.proxy_manager.get_proxy()if not self._current_proxy:raise RuntimeError("No live proxies
available")ua = get_random_ua()self._browser = await self._playwright.chromium.launch(
headless=self.config.headless,proxy=self._current_proxy.to_playwright_dict(),args=[
"--disable-blink-features=AutomationControlled","--no-sandbox","--disable-setuid-sandbox",
"--disable-dev-shm-usage","--disable-gpu","--window-size=1366,768",
"--disable-background-networking","--disable-default-apps","--disable-sync","--mute-audio",])
self._context = await self._browser.new_context(viewport={"width": 1366, "height": 768},
user_agent=ua,locale="en-US",timezone_id=random.choice(["America/New_York",

```

```

"America/Chicago", "America/Denver", "America/Los_Angeles"]),extra_http_headers={
"Accept-Language": "en-US,en;q=0.9","Accept-Encoding": "gzip, deflate, br","sec-ch-ua":
f'"Chromium";v="131", "Not_A Brand";v="24","sec-ch-ua-mobile": "?0","sec-ch-ua-platform":
"Windows","Upgrade-Insecure-Requests": "1",})# Inject navigator.webdriver overrideawait
self._context.add_init_script("""Object.defineProperty(navigator, 'webdriver', {get: () => undefined});
Object.defineProperty(navigator, 'plugins', {get: () => {const arr = [1,2,3,4,5];arr.__proto__ =
PluginArray.prototype;return arr;}});Object.defineProperty(navigator, 'languages', {get: () =>
['en-US','en']});window.chrome = {runtime: {}, loadTimes: function(){}, csi: function(){}, app: {}};
Object.defineProperty(navigator, 'permissions', {get: () => ({query: (p) => Promise.resolve({state:
p.name === 'notifications' ? 'denied' : 'granted'})})});""")self._requests_on_context = 0
self._max_requests_per_context = random.randint(45, 75)logger.info(f"New browser context:
proxy={self._current_proxy.server},ua={ua[:50]}...")async def _maybe_rotate_context(self):
"""Rotate browser context after N requests to avoid fingerprint accumulation."""if
self._requests_on_context >= self._max_requests_per_context:logger.info(f"Rotating context after
{self._requests_on_context} requests")await self._new_browser()async def _cleanup(self):if
self._context:try: await self._context.close()except: passif self._browser:try: await
self._browser.close()except: passasync def fetch(self,url: str,extract_fn: Callable[[Page], Any],
domain: str = "google.com",wait_until: str = "networkidle",) -> Any:"""Navigate to URL, handle
CAPTCHAs, apply extract_fn, return result.Retries with exponential backoff on failure."""last_error
= Nonefor attempt in range(self.config.max_retries):await self._maybe_rotate_context()page =
await self._context.new_page()page.set_default_timeout(self.config.page_timeout)
page.set_default_navigation_timeout(self.config.navigation_timeout)try:# Load session cookies if
available and freshif self.session_mgr.session_age_hours(self._current_proxy.server,domain) < 4:
await self.session_mgr.load_cookies(self._context,self._current_proxy.server,domain)await
page.goto(url, wait_until=wait_until)await HumanDelay.wait("page_load")# CAPTCHA checkif
self.captcha_handler.captcha_type = await self.captcha_handler.detect_captcha(page)if
captcha_type == "recaptcha":sitekey = await self.captcha_handler.get_sitekey(page)if sitekey:
token = await self.captcha_handler.solve_recaptcha_v2(url, sitekey)if token:await
self.captcha_handler.inject_recaptcha_token(page,token)await HumanDelay.wait("page_load")#
Extract dataresult = await extract_fn(page)# Persist cookiesawait
self.session_mgr.save_cookies(self._context,self._current_proxy.server,domain)await
self.proxy_manager.mark_success(self._current_proxy)self._requests_on_context += 1await
HumanDelay.wait("between_pages")return resultexcept Exception as e:last_error = e
logger.warning(f"Attempt {attempt+1}/{self.config.max_retries} failed for {url}: {e}")await
self.proxy_manager.mark_failure(self._current_proxy,str(e))# Exponential backoff with jitterbackoff
= min(self.config.base_backoff * (2 ** attempt) + random.uniform(0, 1),self.config.max_backoff)
await asyncio.sleep(backoff)# Rotate to fresh browser on repeated failuresif attempt >= 1:await
self._new_browser()finally:try:await page.close()except:passraise RuntimeError(f"All
{self.config.max_retries} attempts failed for {url}. Last error: {last_error}")

```

Table 2 — Open-Source Google Maps Scraper GitHub Audit

Repo	Stars	Language	Last Commit	Fit Score	Verdict
gosom/google-maps-scraper	~3,500	Go	Active 2024-2025	6/10	Reference only
gaspa93/googlemaps-scraper	~499	Python	August 2024	2/10	Skip
D3vd/google_maps_scraper	N/A	Unknown	N/A — repo deleted/never existed	N/A	Does not exist — Skip
omkarcloud/google-maps-scraper	~2,600	Python + Node.js	March 2026 (active)	7/10	Fork/reference
christivn/mapScraper	~200	Python	2024	7/10	Fork
ssecgroup/google-maps-scraper-pro	Low	Python	2024-2025	6/10	Reference

Detailed evaluations:

- gosom/google-maps-scraper — Playwright (Rod browser) + Go concurrency. Extracts 33+ fields. Docker + Kubernetes support. REST API + Web UI. Email harvesting built-in. CSV/JSON/PostgreSQL output. Cannot be extended with Python pipeline. Study its scrolling/pagination logic and field extraction patterns.
- gaspa93/googlemaps-scraper — Selenium + ChromeDriver. Focused on review scraping for specific POI URLs — not a search/discovery tool. Requires pre-known URLs. MongoDB integration. Wrong use case for ZIP-level search scraping.
- D3vd/google_maps_scraper — Repo appears deleted or never existed under this path. D3vd is known for Meme_Api and Trivia_OCR_Search. No Google Maps scraper found.
- omkarcloud/google-maps-scraper — Botasaurus framework (anti-detect browser wrapper). Extracts 50+ fields including emails, social profiles. Has a Pro paid tier. Requires Node.js 18+ alongside Python 3.8+. Dual runtime dependency; Botasaurus is a black box; Pro features paywalled.
- christivn/mapScraper — Pure Python + Playwright. Headless Chromium. Extracts place ID, name, category, address, contact, coordinates, ratings, website. Supports language/country params. No built-in proxy rotation or stealth. Good starting skeleton.
- ssecgroup/google-maps-scraper-pro — Python, fault-tolerant, checkpoint-based recovery, zero RAM streaming, deduplication built-in. Good architecture for long-running jobs. Less

Build Recommendation: Build your own, using [christivn/mapScraper](#) as skeleton + [omkarcloud](#) field extraction patterns. None of the existing Python scrapers include production-grade proxy rotation + stealth + async job queue. You need tight integration with the email/phone discovery pipeline (Section 2). [christivn/mapScraper](#) gives you the Playwright navigation logic without the bloat; [omkarcloud](#) shows which fields are extractable and how to parse them; [gosom \(Go\)](#) shows the right architecture for scale — replicate it in Python.

1E. Extractable Fields Per Source

Table 3 — Extractable Fields by Source with Reliability Ratings

Field	Google Maps	Yelp	Yellow Pages
Business Name	Reliable	Reliable	Reliable
Phone Number	Reliable	Reliable	Reliable
Street Address	Reliable	Reliable	Reliable
City/State/ZIP	Reliable	Reliable	Reliable
Website URL	Reliable	~70% present	~60% present
Business Category	Reliable	Reliable	Reliable
Rating (stars)	Reliable	Reliable	~50% present
Review Count	Reliable	Reliable	~40% present
Latitude/Longitude	From URL	Not exposed	Not exposed
Business Hours	Reliable	On detail page	~60% present
Owner Name	~30% (Q&A/posts)	~40% (Meet the Owner)	Rare
Email Address	Not on Maps	Not on Yelp	Not on YP
Social Media Links	Not on Maps	~20%	Rare
Years in Business		~30%	~40%
Number of Employees			~20%
License/Accreditation		~25%	~30%
Photos Count	Reliable	Reliable	
Plus Code	Reliable		
Price Range	~40%	Reliable	
Claimed Status	Reliable	Reliable	~50%

Key insight: No source exposes email directly. Email discovery requires the website URL

SECTION 2 — EMAIL & PHONE DISCOVERY ENGINE

2A. Full Discovery Stack (Waterfall Approach)

The waterfall runs methods in order of reliability/cost. Stop as soon as a verified email is found.

Table 4 — Email Discovery Waterfall: Priority, Method, and Hit Rates

Priority	Method	Hit Rate	Notes
1	Website contact page crawler	~35%	Highest reliability, zero cost
2	SMTP pattern generation + verify	~25% additional	Requires owner name for best results
3	WHOIS registrant lookup	~8% additional	Many domains use privacy protection
4	Social media profile scraping	~5% additional	Facebook About pages; LinkedIn via DDG
5	DuckDuckGo search discovery	~7% additional	No API key required; 1 req/2s rate limit
~80% Total Coverage (businesses with websites)	5 Discovery Methods	Priority 1 Website Crawler (35% hit rate)	\$0.0012 captcha Cost per Solve

Method 1: Website Contact Page Crawler

```
import re
import asyncio
import aiohttp
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse
from typing import Optional

# Comprehensive email regex — handles most real-world formats
EMAIL_REGEX = re.compile(r'\b[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}\b')

# Pages most likely to contain contact info — ordered by hit probability
CONTACT_PAGE_PATHS = [
    "/contact", "/contact-us", "/contact_us", "/contactus", "/about", "/about-us", "/about_us", "/team",
    "/our-team", "/staff", "/reach-us", "/get-in-touch", "/info", "/support", "/help", "/", # Homepage last — often
    # has footer email
]
# Domains to exclude from email results (generic/spam)
EXCLUDED_EMAIL_DOMAINS = {"example.com", "test.com", "domain.com", "email.com",
    "sentry.io", "sentry-next.io", "wixpress.com", "squarespace.com", "wordpress.com", "godaddy.com",
    "amazonaws.com", "cloudflare.com", "google.com", "facebook.com", "twitter.com", "instagram.com",
    "yelp.com", "yellowpages.com", "bbb.org", "angieslist.com"}, # Generic role addresses — lower
```

```

confidence      but still valid
GENERIC_LOCAL_PARTS = {"info", "contact", "hello", "admin", "support",
                        "sales", "office", "mail", "email", "service", "help", "team", "inquiry", "inquiries", "quote", "estimates",}

async def crawl_website_for_emails(website_url: str, session: aiohttp.ClientSession, timeout: int =
10, max_pages: int = 6,) -> list[dict]: """Crawl a business website for email addresses. Returns list of
dicts: {email, source_url, local_part_type, confidence}""" if not website_url.startswith(("http://",
"https://")): website_url = "https://" + website_urlbase_domain =
urlparse(website_url).netloc.lower().lstrip("www.")found_emails: dict[str, dict] = {} # email ->
metadatatavisited_urls: set[str] = set()headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36", "Accept":
"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8", "Accept-Language":
"en-US,en;q=0.9",}async def fetch_page(url: str) -> Optional[str]: if url in visited_urls: return None
visited_urls.add(url)try: async with session.get(url, headers=headers,
timeout=aiohttp.ClientTimeout(total=timeout), allow_redirects=True, ssl=False) as resp: if
resp.status == 200 and "text/html" in resp.headers.get("content-type", ""): return await
resp.text(errors="replace")except Exception: passreturn Nonedef extract_emails_from_html(html:
str, source_url: str) -> list[dict]: results = []soup = BeautifulSoup(html, "lxml")# 1. Check mailto: links
first — highest confidencefor a in soup.find_all("a", href=True): href = a["href"]if
href.startswith("mailto:"): email = href[7:].split("?")[0].strip().lower()if is_valid_email(email,
base_domain): results.append({"email": email, "source_url": source_url, "discovery_method":
"mailto_link", "confidence": 0.95,})# 2. Regex scan of visible texttext = soup.get_text(separator=" ")
for match in EMAIL_REGEX.finditer(text): email = match.group(0).lower().strip(".")if
is_valid_email(email, base_domain) and email not in {r["email"] for r in results}: local_part =
email.split("@")[0]confidence = 0.85 if local_part in GENERIC_LOCAL_PARTS else 0.90
results.append({"email": email, "source_url": source_url, "discovery_method": "regex_text",
"confidence": confidence,})# 3. Scan raw HTML (catches obfuscated emails in JS strings, data
attrs)for match in EMAIL_REGEX.finditer(html): email = match.group(0).lower().strip(".")if
is_valid_email(email, base_domain) and email not in {r["email"] for r in results}: results.append({
"email": email, "source_url": source_url, "discovery_method": "regex_html", "confidence": 0.80,})
return resultsdef is_valid_email(email: str, site_domain: str) -> bool: if not
EMAIL_REGEX.match(email): return Falsedomain = email.split("@")[1].lower()if domain in
EXCLUDED_EMAIL_DOMAINS: return False# Prefer emails on the business's own domain# but
also accept generic domains (gmail, yahoo) — owner may use personal emailif len(email) > 100 or
len(email) < 6: return False# Filter obvious false positivesif any(x in email for x in ["@2x", ".png",
".jpg", ".gif", ".svg", ".css", ".js"]): return Falsereturn True# Phase 1: Try known contact page paths
pages_checked = 0for path in CONTACT_PAGE_PATHS: if pages_checked >= max_pages: break
url = urljoin(website_url, path)html = await fetch_page(url)if html: emails =
extract_emails_from_html(html, url)for e in emails: if e["email"] not in found_emails:
found_emails[e["email"]] = epages_checked += 1if found_emails: # Found emails — check one
more page then stopif pages_checked >= 2: breakawait asyncio.sleep(0.3)# Phase 2: If nothing
found, crawl internal links from homepageif not found_emails and pages_checked > 0:

```

```

homepage_html      = await fetch_page(website_url)if homepage_html:soup = BeautifulSoup(homepage_html,
"lxml")internal_links = []for a in soup.find_all("a", href=True):href = urljoin(website_url, a["href"])
parsed = urlparse(href)if parsed.netloc.lstrip("www.") == base_domain and href not in visited_urls:#
Prioritize contact-related linkslink_text = a.get_text().lower()if any(kw in link_text or kw in
href.lower())for kw in ["contact", "about", "team", "reach", "touch"]):internal_links.insert(0, href)else:
internal_links.append(href)for link in internal_links[:4]:html = await fetch_page(link)if html:emails =
extract_emails_from_html(html, link)for e in emails:if e["email"] not in found_emails:
found_emails[e["email"]] = eawait asyncio.sleep(0.3)return list(found_emails.values())

```

Method 2: SMTP Pattern Generation + Direct Verification

Concept: Given a business name and owner name, generate candidate email addresses, resolve the domain's MX records, then use SMTP RCPT TO to verify existence without sending mail.

```

def generate_email_candidates(owner_first: str,owner_last: str,business_name: str,domain: str,) ->
list[tuple[str, float]]:
    """Returns list of (email, prior_probability) tuples.Probabilities based on
    empirical SMB email pattern analysis."""
    f = owner_first.lower().strip()l = owner_last.lower().strip()f1
    = f[0] if f else ""l1 = l[0] if l else ""# Sanitize business name for email usebiz = re.sub(r'^a-z0-9', "",
    business_name.lower())[:20]candidates = []# Owner-specific patterns (higher confidence if owner
    name known)if f and l:candidates += [(f"{f}@{domain}", 0.18), # john@({f}{l})@{domain}", 0.22), #
    john.smith@({f}{f1})@{domain}", 0.12), # johnsmith@({f1}{l})@{domain}", 0.15), # jsmith@
    ({f1}{l})@{domain}", 0.10), # j.smith@({f}{l})@{domain}", 0.08), # smith@({f}{l})@{domain}", 0.06),
    # smith.john@({f}{f1})@{domain}", 0.05), # smithj@]# Generic business patterns (always try these)
    candidates += [(f"info@{domain}", 0.35), # Most common for SMBs(f"contact@{domain}", 0.20),
    (f"office@{domain}", 0.15),(f"hello@{domain}", 0.12),(f"admin@{domain}", 0.10),
    (f"owner@{domain}", 0.08),(f"sales@{domain}", 0.08),(f"service@{domain}", 0.07),
    (f"support@{domain}", 0.06),(f"mail@{domain}", 0.05),]# Business name patternsif biz:candidates
    += [(f"{biz}@{domain}", 0.06),(f"info@{biz}.com", 0.04), # Try .com variant if domain differs]#
    Deduplicate while preserving orderseen = set()result = []for email, prob in candidates:if email not in
    seen and "@" in email:seen.add(email)result.append((email, prob))return result

```

Success rates by pattern (empirical SMB data):

- info@domain — 35% of SMBs use this as primary
- firstname.lastname@domain — 22% when owner name known
- firstname@domain — 18%
- contact@domain — 20%
- flastname@domain — 15%

Failure modes:

1. Catch-all domains — server accepts RCPT TO for any address. Detection: send to definitely-does-not-exist-xyz123@domain — if accepted, it's catch-all.
2. Greylisting — server temporarily rejects (451) to filter spam. Mitigation: retry after 5 minutes.
3. Connection refused — no MX record or firewall blocks port 25. Mitigation: skip domain, mark

as unverifiable.

4. SMTP banner blocking — server rejects based on reverse DNS of connecting IP. Mitigation: use a clean IP with proper rDNS, or use a VPS with PTR record set.
5. Rate limiting — server blocks after 3-5 RCPT TO attempts. Mitigation: max 3 attempts/domain/hour, spread across time.

Method 3: WHOIS Lookup

```
import whois # pip install python-whoisdef whois_email_lookup(domain: str) -> list[dict]:"""Extract registrant email from WHOIS data.Hit rate: ~15% (many domains use privacy protection)."""results = []try:w = whois.whois(domain)emails = w.emails if isinstance(w.emails, list) else ([w.emails] if w.emails else [])for email in emails:if email and "@" in email:email = email.lower().strip()# Filter WHOIS privacy servicesprivacy_indicators = ["whoisguard", "privacyprotect", "domainsbyproxy", "contactprivacy", "withheldforprivacy", "registrar-abuse","abuse@", "noreply@", "privacy@",]if not any(p in email for p in privacy_indicators):results.append({"email": email,"source": "whois", "confidence": 0.70,"registrant_name": str(w.name or ""),"registrant_org": str(w.org or ""),})except Exception:passreturn results
```

Limitations: GDPR/CCPA compliance has pushed most registrars to redact WHOIS data.

GoDaddy, Namecheap, Google Domains all use privacy protection by default. Effective on older domains (pre-2018) and some budget registrars. Registrant name from WHOIS is often the owner's real name — valuable even when email is redacted.

Method 4: Social Media Profile Scraping Strategy

LinkedIn (highest value, hardest to scrape):

- Direct scraping violates ToS and triggers aggressive bot detection
- Practical approach: Google/DuckDuckGo search site:linkedin.com/in "company name" "pest control" "owner OR founder OR president"
- Extract name from search snippet, then use name for SMTP pattern generation
- Do NOT attempt to scrape LinkedIn directly — IP bans are permanent

Facebook Business Pages:

- Many SMBs list email on their Facebook page (About section)
- URL pattern: https://www.facebook.com/pg/{page_name}/about/
- Use requests + BeautifulSoup; Facebook SSR renders About section
- Failure mode: Facebook increasingly requires login for About pages

```
async def scrape_facebook_about(business_name: str,session: aiohttp.ClientSession,) -> list[dict]:"""Search for Facebook page and extract contact info from About section."""results = []# Step 1: Find Facebook page URL via DuckDuckGofrom duckduckgo_search import DDGSwith DDGS() as ddgs:search_results = list(ddgs.text(f'site:facebook.com "{business_name}" about',max_results=3))fb_urls = [r["href"] for r in search_results if "facebook.com" in r.get("href", "")]if not fb_urls:return resultsfor fb_url in fb_urls[:2]:# Normalize to /about URLabout_url = fb_url.rstrip("/") + "/about" if
```

```

"/about" not in fb_url else fb_url):try:async with session.get(about_url,
timeout=aiohttp.ClientTimeout(total=10)) as resp:html = await resp.text(errors="replace")emails =
EMAIL_REGEX.findall(html)for email in emails:email = email.lower()if "@" in email and
"facebook.com" not in email:results.append({'email': email,"source": "facebook_about",
"confidence": 0.75,})except Exception:passreturn results

```

Method 5: DuckDuckGo Search Email Discovery

```

from duckduckgo_search import DDGS # pip install duckduckgo-searchimport reimport timedef
ddg_email_search(business_name: str,domain: str = "",location: str = "",max_results: int = 10,) ->
list[dict]:""Use DuckDuckGo to find emails associated with a business.No API key required. Rate
limit: ~1 req/2s to avoid 202 responses.""results = []queries = []if domain:
queries.append(f'"{business_name}" email site:{domain}')queries.append(f'"{business_name}"
contact "@{domain}"')queries.append(f'"{business_name}" {location} email contact')
queries.append(f'"{business_name}" owner email')seen_emails = set()with DDGS() as ddgs:for
query in queries[:3]: # Limit to 3 queries per business:search_results = list(ddgs.text(query,
max_results=max_results))for r in search_results:text = r.get("body", "") + " " + r.get("title", "")for
match in EMAIL_REGEX.finditer(text):email = match.group(0).lower().strip(".")if email not in
seen_emails and is_plausible_business_email(email):seen_emails.add(email)results.append({'
'email': email,"source": "duckduckgo_search","source_url": r.get("href", ""),"confidence": 0.65,})
time.sleep(2.0) # DDG rate limitexcept Exception as e:if "202" in str(e) or "Ratelimit" in str(e):
time.sleep(10)breakreturn resultsdef is_plausible_business_email(email: str) -> bool:""Filter out
obvious non-business emails from search results.""domain = email.split("@")[1] if "@" in email
else ""excluded = {"example.com", "test.com", "sentry.io", "github.com", "npmjs.com", "pypi.org",
"w3.org", "schema.org",}if domain in excluded:return Falseif len(email) > 80:return Falseif
re.search(r'\.(png|jpg|gif|svg|css|js|php|html)$', email):return Falsereturn True

```

2B. Complete SMTP Verifier

```

import asyncioimport dns.resolverimport smtplibimport socketimport timeimport hashlibimport json
import loggingfrom dataclasses import dataclass, fieldfrom enum import Enumfrom typing import
Optionalfrom pathlib import Pathfrom collections import defaultdictlogger =
logging.getLogger(__name__)class EmailStatus(Enum):VALID = "valid" # RCPT TO accepted, not
catch-allINVALID = "invalid" # RCPT TO rejected (550/551/553)CATCH_ALL = "catch_all" #
Domain accepts all addressesUNVERIFIABLE = "unverifiable" # Can't connect / no MX / timeout
RATE_LIMITED = "rate_limited" # Hit domain rate limitGREYLISTED = "greylisted" # Temporary
rejection (451)@dataclassclass VerificationResult:email: strstatus: EmailStatusconfidence_score:
float # 0.0 - 1.0method: str # How it was verifiedmx_host: Optional[str] = Nonesmtp_response:
Optional[str] = Noneis_catch_all: bool = Falseverified_at: float = field(default_factory=time.time)def
to_dict(self) -> dict:return {"email": self.email,"status": self.status.value,"confidence_score":
self.confidence_score,"method": self.method,"mx_host": self.mx_host,"smtp_response":

```

```

self.smtp_response, "is_catch_all": self.is_catch_all,"verified_at": self.verified_at,}class
SMTPVerifier: """Production-ready async SMTP email verifier.Features:- MX record lookup via
dnspython (https://github.com/rthalley/dnspython)- SMTP RCPT TO verification with timeout
handling- Catch-all detection via canary address- Per-domain rate limiting (max 3
attempts/domain/hour)- Result caching (disk-backed JSON)- Async support for concurrent
verification- Exponential backoff on greylisting"""HELO_DOMAIN = "mail.firstknock.io" # Use your
actual sending domainFROM_ADDRESS = "verify@firstknock.io"SMTP_TIMEOUT = 10 # seconds
MAX_ATTEMPTS_PER_DOMAIN_PER_HOUR= 3CACHE_TTL_HOURS = 72 # Cache results
for 3 daysdef __init__(self, cache_path: str = ".smtp_cache.json"):self.cache_path =
Path(cache_path)self._cache: dict[str, dict] = self._load_cache()self._domain_attempts: dict[str,
list[float]] = defaultdict(list)self._lock = asyncio.Lock()self._mx_cache: dict[str, list[str]] = {}def
_load_cache(self) -> dict:if self.cache_path.exists():try:with open(self.cache_path) as f:return
json.load(f)except Exception:passreturn {}def _save_cache(self):with open(self.cache_path, "w")
as f:json.dump(self._cache, f)def _cache_key(self, email: str) -> str:return
hashlib.md5(email.lower().encode()).hexdigest()def _get_cached(self, email: str) ->
Optional[VerificationResult]key = self._cache_key(email)if key in self._cache:cached =
self._cache[key]age_hours = (time.time() - cached["verified_at"]) / 3600if age_hours <
self.CACHE_TTL_HOURS:return VerificationResult(email=cached["email"],
status=EmailStatus(cached["status"]),confidence_score=cached["confidence_score"],
method=cached["method"] + "_cached",mx_host=cached.get("mx_host"),
smtp_response=cached.get("smtp_response"),is_catch_all=cached.get("is_catch_all", False),
verified_at=cached["verified_at"],)return Nonedef _set_cached(self, result: VerificationResult)key =
self._cache_key(result.email)self._cache[key] = result.to_dict()# Periodically save (every 100
entries)if len(self._cache) % 100 == 0:self._save_cache()async def _check_rate_limit(self, domain:
str) -> bool: """Returns True if we can proceed, False if rate limited. """async with self._lock:now =
time.time()hour_ago = now - 3600# Prune old attemptsself._domain_attempts[domain] = [t for t in
self._domain_attempts[domain] if t > hour_ago]if len(self._domain_attempts[domain]) >=
self.MAX_ATTEMPTS_PER_DOMAIN_PER_HOURreturn False
self._domain_attempts[domain].append(now)return Trueasync def _resolve_mx(self, domain: str)
-> list[str]: """Resolve MX records, sorted by priority. Returns list of hostnames. """if domain in
self._mx_cache:return self._mx_cache[domain]loop = asyncio.get_event_loop()try:records = await
loop.run_in_executor(None,lambda: dns.resolver.resolve(domain, "MX"))mx_hosts =
[str(r.exchange).rstrip(".") for r in sorted(records, key=lambda x: x.preference)]
self._mx_cache[domain] = mx_hostsreturn mx_hostsexcept (dns.resolver.NXDOMAIN,
dns.resolver.NoAnswer, dns.exception.DNSException):self._mx_cache[domain] = []return []def
_smtp_verify_sync(self,email: str,mx_host: str,check_catch_all: bool = True,) -> tuple[EmailStatus,
str, bool]: """Synchronous SMTP verification (run in executor for async use).Returns (status,
smtp_response, is_catch_all). """domain = email.split("@")[1]is_catch_all = Falsetry:with
smtpplib.SMTP(timeout=self.SMTP_TIMEOUT) as smtp:smtp.connect(mx_host, 25)
smtp.helo(self.HELO_DOMAIN)smtp.mail(self.FROM_ADDRESS)# Check catch-all first with

```

```

canary = addressif check_catch_all:canary = f"definitely-does-not-exist-xk7q2m@{domain}"code, msg
= smtp.rcpt(canary)if code == 250:is_catch_all = True# Still verify the real address — catch-all +
valid = lower confidencecode, msg = smtp.rcpt(email)response = f'{code} {msg.decode('utf-8',
errors='replace')}'if code == 250:return EmailStatus.CATCH_ALL,response, Trueelse:return
EmailStatus.CATCH_ALL,response, True# Verify target emailcode, msg = smtp.rcpt(email)
response = f'{code} {msg.decode('utf-8', errors='replace')}'if code == 250:return
EmailStatus.VALID,response, is_catch_allelif code in (550, 551, 553, 554):return
EmailStatus.INVALID,response, is_catch_allelif code == 451:return EmailStatus.GREYLISTED,
response, is_catch_allelse:return EmailStatus.UNVERIFIABLE, response, is_catch_allexcept
smtpplib.SMTPConnectError as e:return EmailStatus.UNVERIFIABLE, f"ConnectError: {e}", False
except smtpplib.SMTPServerDisconnected as e:return EmailStatus.UNVERIFIABLE,
f"Disconnected: {e}", Falseexcept socket.timeout:return EmailStatus.UNVERIFIABLE, "Timeout",
Falseexcept ConnectionRefusedError:return EmailStatus.UNVERIFIABLE, "ConnectionRefused",
Falseexcept OSError as e:return EmailStatus.UNVERIFIABLE, f"OSError: {e}", Falseasync def
verify(self, email: str) -> VerificationResult:"""Main entry point. Checks cache, rate limits, resolves
MX, runs SMTP verify."""email = email.lower().strip()# 1. Cache checkcached =
self._get_cached(email)if cached:return cacheddomain = email.split("@")[1] if "@" in email else ""if
not domain:return VerificationResult(email=email, status=EmailStatus.INVALID,
confidence_score=0.0, method="format_check")# 2. Rate limit checkif not await
self._check_rate_limit(domain):return VerificationResult(email=email,
status=EmailStatus.RATE_LIMITED,confidence_score=0.0, method="rate_limited")# 3. MX
resolutionmx_hosts = await self._resolve_mx(domain)if not mx_hosts:result = VerificationResult(
email=email, status=EmailStatus.UNVERIFIABLE,confidence_score=0.1, method="no_mx_record")
self._set_cached(result)return result# 4. SMTP verification (run sync in thread pool to avoid
blocking event loop)loop = asyncio.get_event_loop()mx_host = mx_hosts[0]try:status,
smtp_response, is_catch_all = await loop.run_in_executor(None,lambda:
self._smtp_verify_sync(email, mx_host, check_catch_all=True))except Exception as e:status =
EmailStatus.UNVERIFIABLEsmtp_response = str(e)is_catch_all = False# 5. Compute confidence
scoreconfidence = self._compute_confidence(status, is_catch_all, email)result = VerificationResult(
email=email,status=status,confidence_score=confidence,method="smtp_rcpt_to",
mx_host=mx_host,smtp_response=smtp_response[:200] if smtp_response else None,
is_catch_all=is_catch_all,)self._set_cached(result)return resultdef _compute_confidence(self,
status: EmailStatus, is_catch_all: bool, email: str) -> float:base_scores = {EmailStatus.VALID: 0.92,
EmailStatus.CATCH_ALL: 0.45, # Address exists but can't confirmEmailStatus.INVALID: 0.02,
EmailStatus.UNVERIFIABLE: 0.30,EmailStatus.RATE_LIMITED: 0.40,EmailStatus.GREYLISTED:
0.55,}score = base_scores.get(status, 0.30)# Boost for business domain (not gmail/yahoo)domain
= email.split("@")[1]generic_domains = {"gmail.com", "yahoo.com", "hotmail.com", "outlook.com",
"aol.com"}if domain not in generic_domains:score = min(1.0, score + 0.05)# Slight penalty for
generic local partslocal = email.split("@")[0]if local in {"info", "contact", "admin", "support"}:score =
max(0.0, score - 0.03)return round(score, 3)async def verify_batch(self,emails: list[str],

```



```

PhoneValidator: """Validate and classify phone numbers for US businesses."""
TOLL_FREE_PREFIXES = {"800", "833", "844", "855", "866", "877", "888"}
PREMIUM_PREFIXES = {"900", "976"}
@staticmethod
def classify(e164: str) -> dict: """Returns classification metadata for a
US phone number."""
try:
num = phonenumbers.parse(e164, "US")
num_type = phonenumbers.number_type(num)
national = phonenumbers.format_number(num,
PhoneNumberFormat.NATIONAL)
area_code = national.replace("(", "").split(" ")[0].strip()
return {
"e164": e164,
"national": national,
"area_code": area_code,
"is_mobile": num_type ==
phonenumbers.PhoneNumberType.MOBILE,
"is_voip": num_type ==
phonenumbers.PhoneNumberType.VOIP,
"is_toll_free": area_code in
PhoneValidator.TOLL_FREE_PREFIXES,
"is_premium": area_code in
PhoneValidator.PREMIUM_PREFIXES,
"is_valid": phonenumbers.is_valid_number(num),
"carrier": "", # Would need carrier lookup API
}
except Exception as e:
return {"e164": e164, "is_valid": False,
"error": str(e)}

```

2D. Owner Name Extraction

```

import re
import asyncio
import aiohttp
from bs4 import BeautifulSoup
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class OwnerNameResult:
    first_name: str
    last_name: str
    full_name: str
    source: str
    confidence: float
    raw_text: str

    @property
    def email_local_parts(self) -> list[str]:
        """Generate email local part candidates from this name."""
        f = self.first_name.lower()
        l = self.last_name.lower()
        f1 = f[0] if f else ""
        return [f"{f1}.{l}", f"{f}{l}", f"{f1}{l}",
                f"{f1}.{l}", f"{f}{l}", f"{l}", f"{l}.{f}", f"{l}{f1}"]

# Name extraction patterns for website About pages
OWNER_TITLE_PATTERNS = [
    r'(?:(owner|founder|president|ceo|principal|proprietor|managing|(?:(director|partner)))',
    r'\s*[:\-\u2013\u2014]?s*([A-Z][a-z]+(?:[A-Z][a-z]+){1,3})',
    r'([A-Z][a-z]+(?:[A-Z][a-z]+){1,2})\s*[:\-\u2013\u2014]?s*' + r'(?:(owner|founder|president|ceo|principal)',
    r'(?:(meet|about) (?:(the) )?(?:(owner|founder|team)).*?' + r'([A-Z][a-z]+ [A-Z][a-z]+)',
]

NAME_BLACKLIST = {"Contact Us", "About Us", "Our Team", "Meet The", "Learn More", "Read
More", "Click Here", "Get Started", "Free Quote",}

def extract_owner_from_website(html: str, url: str) -> list[OwnerNameResult]:
    """Parse About/Team pages for owner name."""
    results = []
    soup = BeautifulSoup(html, "lxml")
    text = soup.get_text(separator=" ", strip=True)
    for pattern in OWNER_TITLE_PATTERNS:
        for match in re.finditer(pattern, text, re.IGNORECASE):
            name = match.group(1).strip()
            if name in NAME_BLACKLIST or len(name) < 4:
                continue
            parts = name.split()
            if len(parts) >= 2:
                results.append(OwnerNameResult(
                    first_name=parts[0],
                    last_name=parts[-1],
                    full_name=name,
                    source="website_about",
                    confidence=0.70,
                    raw_text=match.group(0),
                ))
    # Also check structured data (JSON-LD)
    for script in soup.find_all("script", type="application/ld+json"):
        try:
            import json
            data = json.loads(script.string)
            if isinstance(data, list):
                data = data[0]
            # LocalBusiness schema may have employee/founder
            for key in ["founder", "employee", "contactPoint"]:
                person = data.get(key, {})
                if isinstance(person, dict) and person.get("name"):
                    name = person["name"]
                    parts = name.split()
                    if len(parts) >= 2:
                        results.append(OwnerNameResult(
                            first_name=parts[0],

```

```

last_name=parts[-1],      full_name=name,source="json_ld_schema",confidence=0.85,))except
Exception:passreturn resultsdef extract_owner_from_google_maps_data(business_data:dict) ->
Optional[OwnerNameResult]:"""Google Maps sometimes shows owner name in:- Owner
responses to reviews ("Response from the owner")- Business posts- Q&A sectionThese are not
reliably extractable via scraping but appear in the DOM."""# Check if owner name was captured
during scrapingowner_raw = business_data.get("owner_name", "")if owner_raw:parts =
owner_raw.strip().split()if len(parts) >= 2:return OwnerNameResult(first_name=parts[0],
last_name=parts[-1],full_name=owner_raw,source="google_maps_owner_field",confidence=0.80,)
return Nonedef extract_owner_from_whois(whois_data: dict) -> Optional[OwnerNameResult]:
"""Extract registrant name from WHOIS data."""name = whois_data.get("registrant_name", "").strip()
if not name or len(name) < 4:return None# Filter out company names (heuristic: no comma, title
case, 2-3 words)parts = name.split()if 2 <= len(parts) <= 3 and all(p[0].isupper() for p in parts if p):#
Looks like a person namereturn OwnerNameResult(first_name=parts[0],last_name=parts[-1],
full_name=name,source="whois_registrant",confidence=0.60,)return Nonedef
score_owner_name_candidates(candidates: list[OwnerNameResult]) -> OwnerNameResult | None:
"""Score and select best owner name from multiple sources.Higher confidence wins; corroboration
across sources boosts score."""if not candidates:return None# Group by normalized full name
name_groups: dict[str, list[OwnerNameResult]] = {}for c in candidates:key =
c.full_name.lower().strip()name_groups.setdefault(key, []).append(c)# Score each unique name
scored = []for name_key, group in name_groups.items():base_confidence = max(c.confidence for c
in group)# Corroboration bonus: +0.10 per additional source confirming same name
corroboration_bonus = min(0.20, (len(group) - 1) * 0.10)final_score = min(1.0, base_confidence +
corroboration_bonus)best = max(group, key=lambda x: x.confidence)best.confidence = final_score
scored.append(best)return max(scored, key=lambda x: x.confidence)

```

Source confidence reference:

Table 5 — Owner Name Source Confidence Scores

Source	Confidence	Notes
google_maps_owner_field	0.80	Direct from Maps listing
json_ld_schema	0.85	Structured data on website
website_about	0.70	Regex from About page text
yelp_meet_owner	0.75	Yelp 'Meet the Owner' section
whois_registrant	0.60	WHOIS (often privacy-protected)
linkedin_search	0.65	Inferred from search snippet
corroboration_bonus	+0.10 per source	Max +0.20 for multi-source confirmation

```

async def extract_yelp_owner(yelp_url: str, session: aiohttp.ClientSession) ->
Optional[OwnerNameResult]: """Yelp business pages sometimes have a 'Meet the Business Owner'
section. Selector: div[data-testid="meet-the-owner"] or section with 'Meet' heading. """ try: async with
session.get(yelp_url, timeout=aiohttp.ClientTimeout(total=10)) as resp: html = await
resp.text(errors="replace") soup = BeautifulSoup(html, "lxml") # Try structured selector
owner_section = soup.find("section", string=re.compile(r"Meet.*Owner", re.I)) if not owner_section:
owner_section = soup.find(attrs={"data-testid": "meet-the-owner"}) if owner_section: # Name is
usually in an h3 or strong within this section name_el = owner_section.find(["h3", "h4", "strong", "b"])
if name_el: name = name_el.get_text(strip=True) parts = name.split() if len(parts) >= 2: return
OwnerNameResult(first_name=parts[0], last_name=parts[-1], full_name=name,
source="yelp_meet_owner", confidence=0.75,) except Exception: pass return None

```

Full Discovery Orchestrator

```

import asyncio
import aiohttp
from typing import Optional
@dataclass
class ContactDiscoveryResult:
    business_id: str
    business_name: str
    domain: str
    emails: list[dict]
    phones: list[dict]
    owner: Optional[OwnerNameResult]
    verification_results: list[VerificationResult]
    best_email: Optional[str]
    best_email_confidence: float
    async def discover_contacts(business: dict, smtp_verifier: SMTPVerifier, session: aiohttp.ClientSession,) -> ContactDiscoveryResult: """Full waterfall contact
discovery for a single business. business dict must have: id, name, website, phone, address,
yelp_url (optional) """ domain = "" if business.get("website"): from urllib.parse import urlparse
parsed = urlparse(business["website"]) domain = parsed.netloc.lstrip("www.").lower()
all_emails: list[dict] = []
owner_candidates: list[OwnerNameResult] = [] # --- Phone ---
phones = [] if business.get("phone"):
normalized = normalize_phone(business["phone"]) if normalized: phones.append({'e164':
normalized, "national": business["phone"], "source": business.get("source", "unknown"),}) # --- Owner
name ---
owner_from_maps = extract_owner_from_google_maps_data(business) if
owner_from_maps: owner_candidates.append(owner_from_maps) # --- Method 1: Website crawler
--- if domain:
website_emails = await crawl_website_for_emails(business["website"], session)
all_emails.extend(website_emails) # Extract owner from website while we're there
try: async with
session.get(f"https://{domain}/about", timeout=aiohttp.ClientTimeout(total=8)) as resp:
if resp.status == 200:
html = await resp.text(errors="replace")
website_owners = extract_owner_from_website(html, f"https://{domain}/about")
owner_candidates.extend(website_owners) except Exception: pass # --- Method 3: WHOIS ---
if domain and not all_emails:
whois_results = whois_email_lookup(domain)
all_emails.extend(whois_results) # Extract owner from WHOIS
for wr in whois_results:
owner_from_whois = extract_owner_from_whois(wr) if owner_from_whois:
owner_candidates.append(owner_from_whois) # --- Method 5: DuckDuckGo ---
if not all_emails:
ddg_results = ddg_email_search(business["name"], domain, location=business.get("address", ""))
all_emails.extend(ddg_results) # --- Method 2: SMTP pattern generation (if owner name known) ---

```

```
best_owner = score_owner_name_candidates(owner_candidates)
if domain and best_owner:
    candidates = generate_email_candidates(best_owner.first_name, best_owner.last_name, business["name"],
    domain,)
# Verify top 5 candidates for email
prior_prob = sorted(candidates, key=lambda x: -x[1])[0:5]
if email not in {e["email"] for e in all_emails}:
    result = await smtp_verifier.verify(email)
if result.status in (EmailStatus.VALID, EmailStatus.CATCH_ALL):
    all_emails.append({"email": email, "source": "smtp_pattern_verify", "confidence": result.confidence_score, "prior_prob": prior_prob,})
# --- Verify all discovered emails ---
verification_results = []
for email_dict in all_emails[:10]: # Cap at 10 per business
    vr = await smtp_verifier.verify(email_dict["email"])
    verification_results.append(vr)
# --- Select best email ---
valid_emails = [(vr, e) for vr, e in zip(verification_results, all_emails) if vr.status in (EmailStatus.VALID, EmailStatus.CATCH_ALL, EmailStatus.UNVERIFIABLE)]
valid_emails.sort(key=lambda x: x[0].confidence_score, reverse=True)
best_email = valid_emails[0][1]["email"] if valid_emails else None
best_confidence = valid_emails[0][0].confidence_score if valid_emails else 0.0
return ContactDiscoveryResult(
    business_id=business.get("id", ""), business_name=business["name"], domain=domain,
    emails=all_emails, phones=phones, owner=best_owner, verification_results=verification_results,
    best_email=best_email, best_email_confidence=best_confidence,)
```

Library Reference

Table 6 — Full Library Reference: 14 Dependencies with Install Commands and GitHub URLs

Library	Purpose	Install	GitHub
playwright	Browser automation	pip install playwright	https://github.com/microsoft/playwright-python
patchright	Stealth Playwright fork	pip install patchright	https://github.com/Kaliiiii-iiii-Vinyzu/patchright-python
beautifulsoup4	HTML parsing	pip install beautifulsoup4	https://www.crummy.com/software/BeautifulSoup/
lxml	Fast HTML parser	pip install lxml	https://github.com/lxml/lxml
aiohttp	Async HTTP client	pip install aiohttp	https://github.com/aio-libs/aiohttp
asyncpg	Async PostgreSQL	pip install asyncpg	https://github.com/MagicStack/asyncpg

dnspython	DNS/MX resolution	pip install dnspython	https://github.com/rthalley/dnspython
python-whois	WHOIS lookups	pip install python-whois	https://github.com/richardpenman/whois
phonenumbers	Phone parsing/E.164	pip install phonenumbers	https://github.com/daviddrysdale/python-phonenumbers
duckduckgo-search	DDG search (no API key)	pip install duckduckgo-search	https://github.com/deedy5/duckduckgo_search
numpy	Normal dist delays	pip install numpy	https://github.com/numpy/numpy
pandas	ZIP code loading	pip install pandas	https://github.com/pandas-dev/pandas
2captcha-python	CAPTCHA solving	pip install 2captcha-python	https://github.com/2captcha/2captcha-python
curl_cffi	TLS fingerprint bypass	pip install curl_cffi	https://github.com/yifeikong/curl_cffi